

AEROJET ACADEMY - COMPLETE PORTAL SYSTEM

TECHNICAL SPECIFICATION FOR DEVELOPERS

Version: 2.0 (Integrated)
Date: February 2026
Document Type: Complete System Specification
Scope: Study Pathways + Wallet + Pooling + All Booking Types

DOCUMENT OVERVIEW

This specification integrates three major system components:

- 1. **Study Pathway Selection & Locking** - First-login wizard, pathway gates
- 2. **Universal Wallet System** - Centralized payment management
- 3. **Exam Pooling System** - Group formation for exam-only students
- 4. **Multi-Pathway Booking Systems** - Full-time, Exam-only, Modular, Tuition

The Complete Student Journey:

Registration → First Login → Choose Pathway (LOCKED) →
→ Fund Wallet → Book Based on Pathway → Attend/Complete

TABLE OF CONTENTS

PART 1: SYSTEM ARCHITECTURE

- 1. Executive Summary
- 2. Study Pathway System
- 3. Universal Wallet System
- 4. Complete Data Model

PART 2: BOOKING SYSTEMS BY PATHWAY 5. Full-Time Programme System 6. Exam-Only System + Pooling 7. Modular Package System 8. Tuition Session System

PART 3: IMPLEMENTATION 9. Business Rules & Validation 10. API Endpoints Complete 11. Wallet Transaction Flows 12. User Interface Specifications 13. Testing & Deployment

1. EXECUTIVE SUMMARY

1.1 The Complete System

Aerojet Academy offers **three distinct study pathways**, each with different booking rules, payment schedules, and student experiences:

Pathway	Description	Payment Model	Key Feature
FULL_TIME	4-year/2-year/1-year programmes	Milestone-based (40/30/30 Year 1, 50/50 thereafter)	Structured curriculum
EXAM_ONLY	Take EASA Part-66 module exams	100% upfront per exam	Can use pooling system (€300/seat)
MODULAR	Purchase pre-packaged module sets	100% upfront per package	Self-paced learning

Plus: All pathways can access **Tuition/Revision Sessions** (100% upfront)

1.2 Critical System Rules

The Pathway Lock:

Once a student chooses their pathway on first login, it is **permanently locked**. Only admins can change it with full audit trail.

The Wallet Principle:

ALL bookings and payments flow through the student's wallet. No direct payment to individual services.

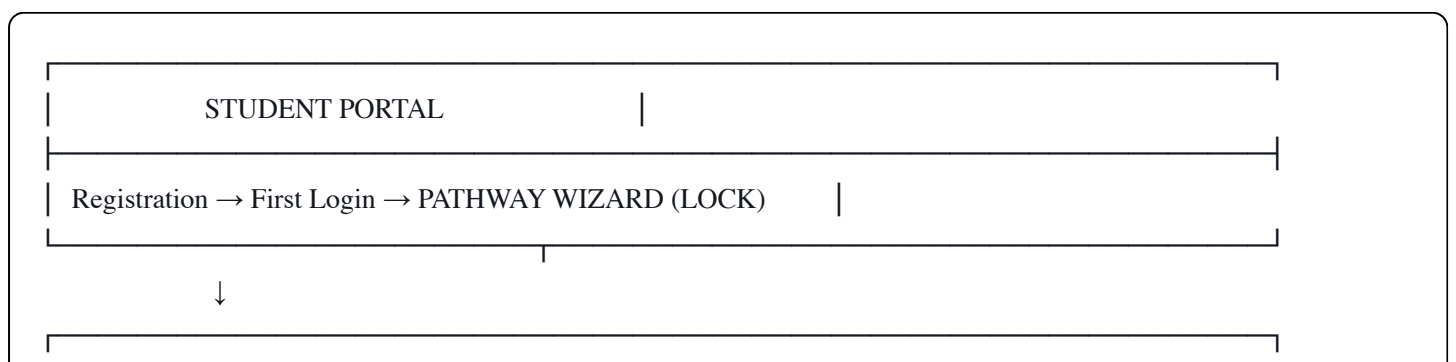
The Pathway Gate:

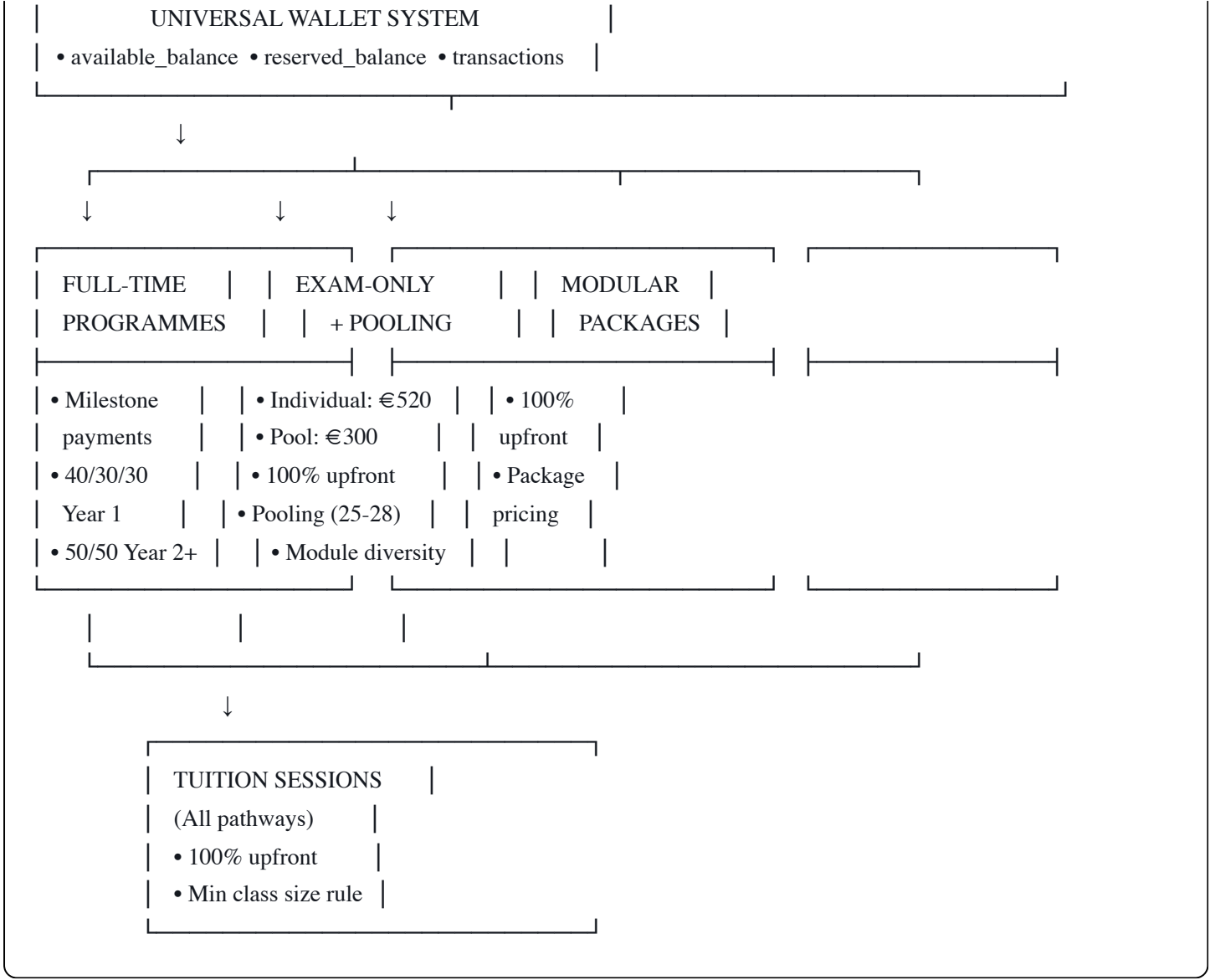
Students cannot book anything until they've chosen a pathway. Wrong-pathway bookings are hard-rejected (403 error).

The Pooling Restriction:

ONLY exam-only students can use the pooling system. Full-time and modular students book individual exam seats.

1.3 Integration Architecture





2. STUDY PATHWAY SYSTEM

2.1 The Pathway Selection Wizard

When: Immediately after first login following registration.

What: Student must choose exactly ONE pathway:

- **FULL_TIME** - Structured multi-year programme
- **EXAM_ONLY** - Individual module exams
- **MODULAR** - Pre-packaged learning modules

Why: Each pathway has completely different:

- Booking workflows
- Payment schedules
- Access to features (pooling, milestones, etc.)
- Student experience

2.2 The Lock Mechanism

Once chosen:

```
javascript

student.study_pathway = 'EXAM_ONLY'
student.study_pathway_locked_at = NOW()
student.study_pathway_locked = TRUE // Cannot be changed by student
```

Implications:

- Student sees only pathway-relevant menus
- Other pathway endpoints return `403 PATHWAY_NOT_ELIGIBLE`
- Permanent decision (like choosing a degree programme)

2.3 Admin Override (Rare Cases)

When Needed:

- Student enrolled in wrong pathway by mistake
- Business exception (e.g., full-time student wants to switch to exam-only)
- System migration/correction

Process:

```
python
```

```

# Admin initiates pathway change
def admin_change_pathway(student_id, new_pathway, reason):
    student = Student.get(student_id)

    # Check for conflicts
    active_bookings = student.get_active_bookings()

    if active_bookings and mode != 'MIGRATION_OVERRIDE':
        return error("Student has active bookings. Use migration mode.")

    # Audit log
    create_audit_log(
        action='PATHWAY_CHANGE',
        student_id=student_id,
        old_pathway=student.study_pathway,
        new_pathway=new_pathway,
        reason=reason,
        admin_user_id=current_admin.id
    )

    # Execute change
    if mode == 'MIGRATE_CANCEL_AND_CREDIT':
        # Cancel all bookings, credit wallet
        for booking in active_bookings:
            cancel_booking(booking)
            credit_wallet(student, booking.amount_paid,
                          reason='Pathway migration')

    # Update pathway
    student.study_pathway = new_pathway
    student.study_pathway_last_changed_by = current_admin.id
    student.study_pathway_change_reason = reason
    student.save()

    # Notify student
    send_email(student, 'PATHWAY_CHANGED', {
        'new_pathway': new_pathway,
        'reason': reason
    })

```

2.4 Pathway Gate Enforcement

Every booking endpoint must validate:

```
python
```

```

def validate_pathway_access(student, required_pathway):
    """
    Enforces pathway-based access control.
    Called at the start of every booking endpoint.
    """
    # Check if pathway chosen
    if not student.study_pathway:
        raise HTTPException(403,
            code='PATHWAY_REQUIRED',
            message='Please choose your study pathway first'
        )

    # Check if pathway matches
    if student.study_pathway != required_pathway:
        raise HTTPException(403,
            code='PATHWAY_NOT_ELIGIBLE',
            message=f'This service requires {required_pathway} pathway'
        )

    return True

# Usage in endpoints
@app.post('/full-time/enrollments')
def create_fulltime_enrollment(student: Student):
    validate_pathway_access(student, 'FULL_TIME')
    # ... proceed with enrollment

```

Specific Rules:

- Full-time endpoints → require `FULL_TIME`
- Exam booking (individual) → require `EXAM_ONLY`
- Pooling endpoints → require `EXAM_ONLY` ⚠️ CRITICAL
- Modular endpoints → require `MODULAR`
- Tuition endpoints → require `EXAM_ONLY` or `MODULAR` (full-time optional)

3. UNIVERSAL WALLET SYSTEM

3.1 The Wallet Concept

Every student has **one wallet** that acts as their prepaid account for ALL Aerojet services.

Balance Components:

Total Balance = Available Balance + Reserved Balance

Available: €500] Can spend on bookings

Reserved: €300] Held for pending pools

Total: €800

Why Wallet?

1. **Single Payment Point:** Student tops up once, books multiple things
2. **Reservation System:** Hold funds for pending confirmations (pooling)
3. **Refund Management:** Credits go back to wallet, not bank account
4. **Accounting Clarity:** All transactions traceable

3.2 Transaction Types

Type	Direction	Effect	Use Case
TOP_UP	IN	+available	Student adds funds via bank transfer
RESERVE	HOLD	available→reserved	Join exam pool (pending confirmation)
CAPTURE	OUT	-available OR -reserved	Booking confirmed, money spent
RELEASE	UNHOLD	reserved→available	Pool failed, return funds
CREDIT	IN	+available	Refund, admin credit
DEBIT	OUT	-available	Admin adjustment (rare)
ADJUSTMENT	IN/OUT	±available	Manual correction

3.3 Wallet Lifecycle Example

Day 1: Student registers

Available: €0

Reserved: €0

Day 2: Student tops up €1,000

Transaction: TOP_UP +€1,000

Available: €1,000

Reserved: €0

Day 3: Student joins Pool A for M1 (€300)

Transaction: RESERVE -€300 (pool pending)

Available: €700

Reserved: €300

Day 5: Pool A confirms (25 candidates reached)

Transaction: CAPTURE -€300 (from reserved)

Available: €700

Reserved: €0

Booking created: CONFIRMED

Day 10: Student joins Pool B for M7 (€300)

Transaction: RESERVE -€300

Available: €400

Reserved: €300

Day 15: Pool B fails (only 18 candidates)

Transaction: RELEASE +€300 (back to available)

Available: €700

Reserved: €0

Day 20: Student books individual exam seat (€520)

Transaction: CAPTURE -€520 (direct from available)

Available: €180

Reserved: €0

Booking created: CONFIRMED

3.4 Wallet Database Schema

sql

```
-- =====  
-- WALLETS TABLE  
-- =====
```

```
CREATE TABLE wallets (  
  id BIGSERIAL PRIMARY KEY,  
  student_id BIGINT NOT NULL UNIQUE REFERENCES students(id),  
  
  -- Balance components  
  available_balance DECIMAL(10,2) DEFAULT 0.00,  
  reserved_balance DECIMAL(10,2) DEFAULT 0.00,  
  
  -- Currency  
  currency VARCHAR(3) DEFAULT 'EUR',  
  
  -- Audit  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW(),  
  
  -- Constraints  
  CONSTRAINT positive_available CHECK (available_balance >= 0),  
  CONSTRAINT positive_reserved CHECK (reserved_balance >= 0)  
);
```

```
CREATE INDEX idx_wallets_student ON wallets(student_id);
```

```
-- =====  
-- WALLET TRANSACTIONS TABLE  
-- =====
```

```
CREATE TABLE wallet_transactions (  
  id BIGSERIAL PRIMARY KEY,  
  wallet_id BIGINT NOT NULL REFERENCES wallets(id),  
  
  -- Transaction metadata  
  type VARCHAR(20) NOT NULL,  
  -- Values: TOP_UP, RESERVE, CAPTURE, RELEASE, CREDIT, DEBIT, ADJUSTMENT  
  
  amount DECIMAL(10,2) NOT NULL,  
  currency VARCHAR(3) DEFAULT 'EUR',  
  
  -- Balance snapshots (audit trail)  
  available_before DECIMAL(10,2),  
  available_after DECIMAL(10,2),  
  reserved_before DECIMAL(10,2),  
  reserved_after DECIMAL(10,2),
```

```

-- Reference (what this relates to)
ref_type VARCHAR(50),
-- Values: REGISTRATION, FULLTIME_MILESTONE, EXAM_BOOKING,
-- MODULAR_PACKAGE, TUITION_SESSION, POOLING_RESERVATION
ref_id BIGINT,

-- Status
status VARCHAR(20) DEFAULT 'POSTED',
-- Values: PENDING, POSTED, VOID

-- Audit
note TEXT,
created_by VARCHAR(100),
created_at TIMESTAMP DEFAULT NOW(),

CONSTRAINT non_zero_amount CHECK (amount != 0)
);

CREATE INDEX idx_wallet_txns_wallet ON wallet_transactions(wallet_id);
CREATE INDEX idx_wallet_txns_type ON wallet_transactions(type);
CREATE INDEX idx_wallet_txns_ref ON wallet_transactions(ref_type, ref_id);
CREATE INDEX idx_wallet_txns_created ON wallet_transactions(created_at DESC);

```

3.5 Top-Up Process

Student Side:

1. Student clicks "Top Up Wallet"
2. Enters amount (minimum €50)
3. System generates payment instructions (bank details, reference code)
4. Student makes bank transfer
5. Uploads proof of payment (screenshot/receipt)

Admin Side:

1. Admin checks bank account
2. Verifies transfer received
3. Matches with student's proof
4. Posts top-up to wallet:

python

```

def admin_post_topup(wallet_id, amount, proof_doc_id, admin_user_id):
    """
    Admin verifies and posts a pending top-up.
    """
    wallet = Wallet.get(wallet_id)

    BEGIN TRANSACTION

    # Create transaction record
    txn = WalletTransaction.create(
        wallet_id=wallet_id,
        type='TOP_UP',
        amount=amount,
        available_before=wallet.available_balance,
        available_after=wallet.available_balance + amount,
        reserved_before=wallet.reserved_balance,
        reserved_after=wallet.reserved_balance,
        ref_type='TOPUP_PROOF',
        ref_id=proof_doc_id,
        status='POSTED',
        created_by=f'admin:{admin_user_id}',
        note=f'Bank transfer verified by admin {admin_user_id}'
    )

    # Update wallet
    wallet.available_balance += amount
    wallet.updated_at = NOW()
    wallet.save()

    COMMIT

    # Notify student
    send_email(wallet.student, 'WALLET_CREDITED', {
        'amount': amount,
        'new_balance': wallet.available_balance
    })

    return txn

```

4. COMPLETE DATA MODEL

4.1 Student Profile Extensions

sql

```

-- =====
-- STUDENTS TABLE (Extensions)
-- =====

ALTER TABLE students
ADD COLUMN IF NOT EXISTS study_pathway VARCHAR(20) NULL;
-- Values: FULL_TIME, EXAM_ONLY, MODULAR
-- NULL until first login wizard completed

ALTER TABLE students
ADD COLUMN IF NOT EXISTS study_pathway_locked_at TIMESTAMP NULL;

ALTER TABLE students
ADD COLUMN IF NOT EXISTS study_pathway_locked BOOLEAN DEFAULT FALSE;

ALTER TABLE students
ADD COLUMN IF NOT EXISTS study_pathway_last_changed_by BIGINT NULL
REFERENCES admin_users(id);

ALTER TABLE students
ADD COLUMN IF NOT EXISTS study_pathway_change_reason TEXT NULL;

-- Index for pathway queries
CREATE INDEX idx_students_pathway ON students(study_pathway)
WHERE study_pathway IS NOT NULL;

```

4.2 Full-Time Programme Tables

```

sql

```

```
-- =====  
-- FULL-TIME PROGRAMMES  
-- =====
```

```
CREATE TABLE full_time_programmes (  
  id BIGSERIAL PRIMARY KEY,  
  name VARCHAR(200) NOT NULL, -- "4-Year Full-Time", "2-Year", "1-Year"  
  duration_years INTEGER NOT NULL, -- 4, 2, or 1  
  year_fee_amount DECIMAL(10,2) NOT NULL, -- Base fee per year  
  description TEXT,  
  active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW(),  
  
  CONSTRAINT valid_duration CHECK (duration_years IN (1, 2, 4))  
);
```

```
-- Seed data
```

```
INSERT INTO full_time_programmes (name, duration_years, year_fee_amount) VALUES  
(  
'4-Year Full-Time Programme', 4, 8000.00),  
(  
'2-Year Full-Time Programme', 2, 10000.00),  
(  
'1-Year Full-Time Programme', 1, 12000.00);
```

```
-- =====  
-- ACADEMIC YEARS (within programmes)  
-- =====
```

```
CREATE TABLE academic_years (  
  id BIGSERIAL PRIMARY KEY,  
  programme_id BIGINT NOT NULL REFERENCES full_time_programmes(id),  
  year_number INTEGER NOT NULL, -- 1, 2, 3, 4  
  
  -- Semester dates  
  semester1_start_date DATE NOT NULL,  
  semester2_start_date DATE NOT NULL,  
  
  -- Can override programme default fee  
  year_fee_amount DECIMAL(10,2),  
  
  active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT NOW(),  
  
  UNIQUE (programme_id, year_number),  
  CONSTRAINT valid_year_number CHECK (year_number >= 1 AND year_number <= 4)  
);
```

```

-- =====
-- FULL-TIME ENROLLMENTS
-- =====

CREATE TABLE full_time_enrollments (
    id BIGSERIAL PRIMARY KEY,
    student_id BIGINT NOT NULL REFERENCES students(id),
    programme_id BIGINT NOT NULL REFERENCES full_time_programmes(id),

    -- Current status
    status VARCHAR(30) DEFAULT 'PENDING_CONFIRMATION',
    -- Values: PENDING_CONFIRMATION, ACTIVE, AT_RISK, SUSPENDED, WITHDRAWN, COMPLETED

    current_year_number INTEGER DEFAULT 1,

    -- Audit
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),

    -- One active enrollment per student
    UNIQUE (student_id, programme_id)
);

CREATE INDEX idx_ft_enrollments_student ON full_time_enrollments(student_id);
CREATE INDEX idx_ft_enrollments_status ON full_time_enrollments(status);

-- =====
-- PAYMENT MILESTONES
-- =====

CREATE TABLE payment_milestones (
    id BIGSERIAL PRIMARY KEY,
    enrollment_id BIGINT NOT NULL REFERENCES full_time_enrollments(id),
    year_number INTEGER NOT NULL,

    -- Milestone type
    milestone_type VARCHAR(30) NOT NULL,
    -- Values: SEAT_CONFIRMATION, SEM1_DUE, SEM2_DUE

    -- Financial
    due_date DATE NOT NULL,
    percent_of_year_fee DECIMAL(5,2) NOT NULL, -- 40.00, 30.00, 50.00
    amount_due DECIMAL(10,2) NOT NULL,

    -- Payment status
    status VARCHAR(20) DEFAULT 'DUE',
    -- Values: DUE, PAID, OVERDUE, WAIVED

```

```
paid_at TIMESTAMP NULL,
wallet_txn_id BIGINT NULL REFERENCES wallet_transactions(id),

created_at TIMESTAMP DEFAULT NOW(),

CONSTRAINT valid_milestone_type CHECK (
    milestone_type IN ('SEAT_CONFIRMATION', 'SEM1_DUE', 'SEM2_DUE')
),
CONSTRAINT valid_percent CHECK (percent_of_year_fee > 0 AND percent_of_year_fee <= 100)
);

CREATE INDEX idx_milestones_enrollment ON payment_milestones(enrollment_id);
CREATE INDEX idx_milestones_status ON payment_milestones(status);
CREATE INDEX idx_milestones_due_date ON payment_milestones(due_date);
```

4.3 Exam-Only Tables (Including Pooling)

sql

```

-- =====
-- EXAM MODULES (Reference Data)
-- =====

CREATE TABLE exam_modules (
  id BIGSERIAL PRIMARY KEY,
  code VARCHAR(10) UNIQUE NOT NULL, -- M1, M2, M7, etc.
  name VARCHAR(200) NOT NULL,
  category VARCHAR(50),
  duration_minutes INTEGER DEFAULT 180,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Seed from earlier (M1-M17)

-- =====
-- EXAM BOOKINGS (Individual, non-pool)
-- =====

CREATE TABLE exam_bookings (
  id BIGSERIAL PRIMARY KEY,
  student_id BIGINT NOT NULL REFERENCES students(id),
  exam_module_id BIGINT NOT NULL REFERENCES exam_modules(id),
  event_window_id BIGINT NULL REFERENCES exam_events(id),

  -- Status
  status VARCHAR(20) DEFAULT 'DRAFT',
  -- Values: DRAFT, CONFIRMED, SCHEDULED, COMPLETED, CANCELLED

  -- Financial
  amount DECIMAL(10,2) NOT NULL,
  wallet_txn_id BIGINT NULL REFERENCES wallet_transactions(id),

  -- Audit
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),

  CONSTRAINT positive_exam_amount CHECK (amount > 0)
);

CREATE INDEX idx_exam_bookings_student ON exam_bookings(student_id);
CREATE INDEX idx_exam_bookings_status ON exam_bookings(status);

-- =====
-- POOLING TABLES (from previous spec)

```

-- =====

-- EXAM EVENTS

```
CREATE TABLE exam_events (  
  id BIGSERIAL PRIMARY KEY,  
  name VARCHAR(200) NOT NULL,  
  start_date DATE NOT NULL,  
  end_date DATE NOT NULL,  
  payment_deadline DATE NOT NULL, -- T-21  
  min_revenue_target DECIMAL(10,2) DEFAULT 25000.00,  
  status VARCHAR(20) DEFAULT 'DRAFT',  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW()  
);
```

-- POOLS

```
CREATE TABLE exam_pools (  
  id BIGSERIAL PRIMARY KEY,  
  event_id BIGINT NOT NULL REFERENCES exam_events(id) ON DELETE CASCADE,  
  name VARCHAR(200) NOT NULL,  
  exam_date DATE NOT NULL,  
  exam_start_time TIME NOT NULL,  
  exam_end_time TIME NOT NULL,  
  min_candidates INTEGER DEFAULT 25,  
  max_candidates INTEGER DEFAULT 28,  
  module_diversity_cap INTEGER DEFAULT 4,  
  status VARCHAR(20) DEFAULT 'DRAFT',  
  pre_seed_modules VARCHAR(100)[],  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW()  
);
```

-- POOL MEMBERSHIPS

```
CREATE TABLE pool_memberships (  
  id BIGSERIAL PRIMARY KEY,  
  pool_id BIGINT NOT NULL REFERENCES exam_pools(id) ON DELETE CASCADE,  
  student_id BIGINT NOT NULL REFERENCES students(id),  
  module_code VARCHAR(10) NOT NULL,  
  status VARCHAR(20) DEFAULT 'RESERVED',  
  price_paid DECIMAL(10,2) NOT NULL,  
  reserved_amount DECIMAL(10,2),  
  discount_type VARCHAR(50),  
  referral_code VARCHAR(50),  
  wallet_txn_id BIGINT NULL REFERENCES wallet_transactions(id),  
  created_at TIMESTAMP DEFAULT NOW(),  
  confirmation_date TIMESTAMP,
```

```
    UNIQUE (pool_id, student_id)
);

-- (Full pooling schema from previous spec - all tables included)
```

4.4 Modular Package Tables

```
sql
```

```
-- =====
-- MODULAR PACKAGES
-- =====
```

```
CREATE TABLE modular_packages (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(200) NOT NULL,
  description TEXT,
  price DECIMAL(10,2) NOT NULL,
  modules_included TEXT[], -- Array of module codes
  duration_months INTEGER, -- Validity period
  active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),

  CONSTRAINT positive_modular_price CHECK (price > 0)
);
```

```
-- Example packages
```

```
INSERT INTO modular_packages (name, description, price, modules_included, duration_months) VALUES
('Foundation 4-Pack', 'Core foundation modules', 1800.00, ARRAY['M1', 'M2', 'M7', 'M8'], 12),
('Powerplant Specialist', 'Gas turbine focus', 1500.00, ARRAY['M13', 'M15', 'M17'], 12);
```

```
-- =====
-- MODULAR ENROLLMENTS
-- =====
```

```
CREATE TABLE modular_enrollments (
  id BIGSERIAL PRIMARY KEY,
  student_id BIGINT NOT NULL REFERENCES students(id),
  package_id BIGINT NOT NULL REFERENCES modular_packages(id),

  -- Status
  status VARCHAR(20) DEFAULT 'DRAFT',
  -- Values: DRAFT, CONFIRMED, ACTIVE, COMPLETED, CANCELLED

  -- Financial
  amount_paid DECIMAL(10,2) NOT NULL,
  wallet_txn_id BIGINT NULL REFERENCES wallet_transactions(id),

  -- Validity
  valid_from DATE,
  valid_until DATE,

  -- Audit
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
```

```
CONSTRAINT positive_modular_amount CHECK (amount_paid > 0)
);

CREATE INDEX idx_modular_enrollments_student ON modular_enrollments(student_id);
CREATE INDEX idx_modular_enrollments_status ON modular_enrollments(status);
```

4.5 Tuition Session Tables

sql

```
-- =====  
-- TUITION RUNS  
-- =====
```

```
CREATE TABLE tuition_runs (  
  id BIGSERIAL PRIMARY KEY,  
  title VARCHAR(200) NOT NULL,  
  module_tag VARCHAR(10), -- e.g., 'M2', 'M7'  
  description TEXT,  
  
  -- Schedule  
  start_datetime TIMESTAMP NOT NULL,  
  end_datetime TIMESTAMP NOT NULL,  
  
  -- Capacity  
  capacity INTEGER NOT NULL,  
  min_class_size INTEGER DEFAULT 5,  
  current_enrollments INTEGER DEFAULT 0,  
  
  -- Financial  
  price DECIMAL(10,2) NOT NULL,  
  
  -- Status  
  status VARCHAR(20) DEFAULT 'SCHEDULED',  
  -- Values: SCHEDULED, OPEN, FULL, CANCELLED, COMPLETED  
  
  -- Audit  
  created_by BIGINT REFERENCES admin_users(id),  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW(),  
  
  CONSTRAINT valid_tuition_capacity CHECK (capacity > 0),  
  CONSTRAINT valid_tuition_time CHECK (end_datetime > start_datetime),  
  CONSTRAINT positive_tuition_price CHECK (price > 0)  
);
```

```
CREATE INDEX idx_tuition_runs_module ON tuition_runs(module_tag);  
CREATE INDEX idx_tuition_runs_status ON tuition_runs(status);  
CREATE INDEX idx_tuition_runs_start ON tuition_runs(start_datetime);
```

```
-- =====  
-- TUITION BOOKINGS  
-- =====
```

```
CREATE TABLE tuition_bookings (  
  id BIGSERIAL PRIMARY KEY,  
  tuition_run_id BIGINT NOT NULL REFERENCES tuition_runs(id),
```

```
student_id BIGINT NOT NULL REFERENCES students(id),

-- Status
status VARCHAR(20) DEFAULT 'DRAFT',
-- Values: DRAFT, CONFIRMED, CANCELLED, ATTENDED, NO_SHOW

-- Financial
amount_paid DECIMAL(10,2) NOT NULL,
wallet_txn_id BIGINT NULL REFERENCES wallet_transactions(id),

-- Audit
created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW(),

UNIQUE (tuition_run_id, student_id),
CONSTRAINT positive_tuition_amount CHECK (amount_paid > 0)
);

CREATE INDEX idx_tuition_bookings_run ON tuition_bookings(tuition_run_id);
CREATE INDEX idx_tuition_bookings_student ON tuition_bookings(student_id);
CREATE INDEX idx_tuition_bookings_status ON tuition_bookings(status);
```

5. FULL-TIME PROGRAMME SYSTEM

5.1 Payment Schedule Rules

Year 1 (All Programmes):

40% Seat Confirmation (due immediately upon enrollment)
30% Semester 1 (due before Semester 1 start date)
30% Semester 2 (due before Semester 2 start date)

Years 2+ (2-year and 4-year only):

50% Semester 1 (due before Semester 1 start date)
50% Semester 2 (due before Semester 2 start date)

5.2 Milestone Generation Algorithm

python

```

def create_full_time_enrollment(student_id, programme_id):
    """
    Creates enrollment and generates all payment milestones.
    """
    programme = FullTimeProgramme.get(programme_id)

    BEGIN TRANSACTION

    # Create enrollment
    enrollment = FullTimeEnrollment.create(
        student_id=student_id,
        programme_id=programme_id,
        status='PENDING_CONFIRMATION', # Cannot be ACTIVE until seat paid
        current_year_number=1
    )

    # Generate milestones for all years
    for year_num in range(1, programme.duration_years + 1):
        academic_year = AcademicYear.get(programme_id, year_num)
        year_fee = academic_year.year_fee_amount or programme.year_fee_amount

        if year_num == 1:
            # Year 1: 40/30/30
            create_milestone(
                enrollment_id=enrollment.id,
                year_number=1,
                milestone_type='SEAT_CONFIRMATION',
                due_date=enrollment.created_at.date(), # Due immediately
                percent=40.00,
                amount=year_fee * 0.40
            )
            create_milestone(
                enrollment_id=enrollment.id,
                year_number=1,
                milestone_type='SEM1_DUE',
                due_date=academic_year.semester1_start_date - timedelta(days=7),
                percent=30.00,
                amount=year_fee * 0.30
            )
            create_milestone(
                enrollment_id=enrollment.id,
                year_number=1,
                milestone_type='SEM2_DUE',
                due_date=academic_year.semester2_start_date - timedelta(days=7),
                percent=30.00,
                amount=year_fee * 0.30
            )

```

```
)
else:
    # Years 2+: 50/50
    create_milestone(
        enrollment_id=enrollment.id,
        year_number=year_num,
        milestone_type='SEM1_DUE',
        due_date=academic_year.semester1_start_date - timedelta(days=7),
        percent=50.00,
        amount=year_fee * 0.50
    )
    create_milestone(
        enrollment_id=enrollment.id,
        year_number=year_num,
        milestone_type='SEM2_DUE',
        due_date=academic_year.semester2_start_date - timedelta(days=7),
        percent=50.00,
        amount=year_fee * 0.50
    )

COMMIT

return enrollment
```

5.3 Paying a Milestone

python

```

def pay_milestone(milestone_id, student_id):
    """
    Student pays a milestone from their wallet.
    """
    milestone = PaymentMilestone.get(milestone_id)
    enrollment = milestone.enrollment
    student = Student.get(student_id)
    wallet = student.wallet

    # Validate
    if milestone.status != 'DUE':
        raise ValueError("Milestone already paid or waived")

    if enrollment.student_id != student_id:
        raise PermissionError("Not your enrollment")

    if wallet.available_balance < milestone.amount_due:
        raise InsufficientFundsError(
            f"Need €{milestone.amount_due}, have €{wallet.available_balance}"
        )

    BEGIN TRANSACTION

    # Create wallet transaction (CAPTURE)
    wallet_txn = WalletTransaction.create(
        wallet_id=wallet.id,
        type='CAPTURE',
        amount=milestone.amount_due,
        available_before=wallet.available_balance,
        available_after=wallet.available_balance - milestone.amount_due,
        reserved_before=wallet.reserved_balance,
        reserved_after=wallet.reserved_balance,
        ref_type='FULLTIME_MILESTONE',
        ref_id=milestone.id,
        status='POSTED',
        note=f'Payment for {milestone.milestone_type} Year {milestone.year_number}'
    )

    # Deduct from wallet
    wallet.available_balance -= milestone.amount_due
    wallet.save()

    # Mark milestone paid
    milestone.status = 'PAID'
    milestone.paid_at = NOW()
    milestone.wallet_txn_id = wallet_txn.id

```

```

milestone.save()

# Special case: Seat confirmation activates enrollment
if milestone.milestone_type == 'SEAT_CONFIRMATION':
    enrollment.status = 'ACTIVE'
    enrollment.save()

COMMIT

# Notify student
send_email(student, 'MILESTONE_PAID', {
    'milestone_type': milestone.milestone_type,
    'amount': milestone.amount_due,
    'year': milestone.year_number
})

# If seat confirmation, send welcome pack
if milestone.milestone_type == 'SEAT_CONFIRMATION':
    send_email(student, 'ENROLLMENT_ACTIVATED', {
        'programme': enrollment.programme.name
    })

return milestone

```

5.4 Enrollment Status Transitions

PENDING_CONFIRMATION → (Seat milestone paid) → ACTIVE
 ACTIVE → (Milestone overdue >30 days) → AT_RISK
 AT_RISK → (Admin action) → SUSPENDED
 SUSPENDED → (Payment made + admin approval) → ACTIVE
 ACTIVE → (Student request) → WITHDRAWN
 ACTIVE → (All years complete) → COMPLETED

6. EXAM-ONLY SYSTEM + POOLING

6.1 Two Exam Booking Options

Option 1: Individual Exam Seat (€520)

- Guaranteed slot
- 100% upfront payment
- Immediate confirmation
- No dependency on other students

Option 2: Pool Seat (€300 base, €270 with discounts)

- Group formation required (25-28 students)
- Reserve funds until pool confirms
- Potential discounts (multi-pool, ambassador)
- Only for EXAM_ONLY pathway students

6.2 Individual Exam Booking Flow

python

```

def book_individual_exam(student_id, module_code, event_id):
    """
    Book individual exam seat (€520, 100% upfront).
    EXAM_ONLY pathway only.
    """
    student = Student.get(student_id)

    # Pathway gate
    if student.study_pathway != 'EXAM_ONLY':
        raise PathwayError("Individual exam booking requires EXAM_ONLY pathway")

    module = ExamModule.get_by_code(module_code)
    price = Decimal('520.00') # Fixed individual pricing
    wallet = student.wallet

    # Validate wallet balance
    if wallet.available_balance < price:
        raise InsufficientFundsError(
            f"Need €{price}, have €{wallet.available_balance}"
        )

    BEGIN TRANSACTION

    # Create booking (DRAFT)
    booking = ExamBooking.create(
        student_id=student_id,
        exam_module_id=module.id,
        event_window_id=event_id,
        status='DRAFT',
        amount=price
    )

    # Capture funds from wallet
    wallet_txn = WalletTransaction.create(
        wallet_id=wallet.id,
        type='CAPTURE',
        amount=price,
        available_before=wallet.available_balance,
        available_after=wallet.available_balance - price,
        ref_type='EXAM_BOOKING',
        ref_id=booking.id,
        status='POSTED',
        note=f'Individual exam seat for {module_code}'
    )

    # Update wallet

```

```

wallet.available_balance -= price
wallet.save()

# Confirm booking
booking.status = 'CONFIRMED'
booking.wallet_txn_id = wallet_txn.id
booking.save()

COMMIT

# Notify student
send_email(student, 'EXAM_BOOKING_CONFIRMED', {
    'module': module.name,
    'amount': price
})

return booking

```

6.3 Pool Booking Integration

The pooling system from the previous specification applies here with one critical addition:

Pathway Enforcement in Every Pool Endpoint:

```

python

# In every pooling endpoint
@app.post('/api/pool-memberships')
def join_pool(pool_id: int, module_code: str, student: Student):
    # CRITICAL: Pathway check FIRST
    if student.study_pathway != 'EXAM_ONLY':
        raise HTTPException(403,
            code='PATHWAY_NOT_ELIGIBLE',
            message='Pooling is only available to EXAM_ONLY students'
        )

    # ... rest of pooling logic from previous spec

```

Wallet Integration:

- Pool join → RESERVE funds (not CAPTURE)
- Pool confirms → CAPTURE from reserved
- Pool fails → RELEASE back to available

[Full pooling system from previous specification applies here with wallet integration]

7. MODULAR PACKAGE SYSTEM

7.1 Package Selection and Purchase

python

```

def purchase_modular_package(student_id, package_id):
    """
    Student purchases modular package (100% upfront).
    MODULAR pathway only.
    """
    student = Student.get(student_id)

    # Pathway gate
    if student.study_pathway != 'MODULAR':
        raise PathwayError("Modular packages require MODULAR pathway")

    package = ModularPackage.get(package_id)

    if not package.active:
        raise ValueError("Package not available")

    wallet = student.wallet

    # Validate balance
    if wallet.available_balance < package.price:
        raise InsufficientFundsError(
            f"Need €{package.price}, have €{wallet.available_balance}"
        )

    BEGIN TRANSACTION

    # Create enrollment (DRAFT)
    enrollment = ModularEnrollment.create(
        student_id=student_id,
        package_id=package_id,
        status='DRAFT',
        amount_paid=package.price,
        valid_from=date.today(),
        valid_until=date.today() + timedelta(days=package.duration_months * 30)
    )

    # Capture funds
    wallet_txn = WalletTransaction.create(
        wallet_id=wallet.id,
        type='CAPTURE',
        amount=package.price,
        available_before=wallet.available_balance,
        available_after=wallet.available_balance - package.price,
        ref_type='MODULAR_PACKAGE',
        ref_id=enrollment.id,
        status='POSTED',
    )

```

```

    note=f'Modular package: {package.name}'
)

# Update wallet
wallet.available_balance -= package.price
wallet.save()

# Activate enrollment
enrollment.status = 'CONFIRMED'
enrollment.wallet_txn_id = wallet_txn.id
enrollment.save()

COMMIT

# Grant access to materials
grant_module_access(student, package.modules_included)

# Notify student
send_email(student, 'MODULAR_PACKAGE_PURCHASED', {
    'package_name': package.name,
    'modules': package.modules_included,
    'valid_until': enrollment.valid_until
})

return enrollment

```

8. TUITION SESSION SYSTEM

8.1 Tuition Booking Rules

Who can book:

- EXAM_ONLY students:  Always
- MODULAR students:  Always
- FULL_TIME students:  Optional (configurable per session)

Payment: 100% upfront from wallet

Minimum Class Size Rule: If tuition run doesn't reach min_class_size by cutoff date (e.g., T-7):

1. Run status → CANCELLED
2. All bookings → CANCELLED
3. Funds returned to wallet (CREDIT transaction)
4. Students notified

8.2 Tuition Booking Flow

python

```

def book_tuition_session(student_id, tuition_run_id):
    """
    Book tuition/revision session (100% upfront).
    Available to EXAM_ONLY and MODULAR students.
    """

    student = Student.get(student_id)
    tuition_run = TuitionRun.get(tuition_run_id)

    # Pathway gate
    if student.study_pathway not in ['EXAM_ONLY', 'MODULAR']:
        # Check if full-time students allowed for this run
        if not tuition_run.allow_fulltime_students:
            raise PathwayError(
                "Tuition sessions available to EXAM_ONLY and MODULAR students only"
            )

    # Validate run status
    if tuition_run.status not in ['OPEN', 'SCHEDULED']:
        raise ValueError(f"Tuition run is {tuition_run.status}")

    # Check capacity
    if tuition_run.current_enrollments >= tuition_run.capacity:
        raise CapacityError("Tuition session is full")

    wallet = student.wallet

    # Validate balance
    if wallet.available_balance < tuition_run.price:
        raise InsufficientFundsError(
            f"Need €{tuition_run.price}, have €{wallet.available_balance}"
        )

    BEGIN TRANSACTION

    # Create booking (DRAFT)
    booking = TuitionBooking.create(
        tuition_run_id=tuition_run_id,
        student_id=student_id,
        status='DRAFT',
        amount_paid=tuition_run.price
    )

    # Capture funds
    wallet_txn = WalletTransaction.create(
        wallet_id=wallet.id,
        type='CAPTURE',

```

```

    amount=tuition_run.price,
    available_before=wallet.available_balance,
    available_after=wallet.available_balance - tuition_run.price,
    ref_type='TUITION_SESSION',
    ref_id=booking.id,
    status='POSTED',
    note=f'Tuition: {tuition_run.title}'
)

# Update wallet
wallet.available_balance -= tuition_run.price
wallet.save()

# Confirm booking
booking.status = 'CONFIRMED'
booking.wallet_txn_id = wallet_txn.id
booking.save()

# Update run enrollment count
tuition_run.current_enrollments += 1
if tuition_run.current_enrollments >= tuition_run.capacity:
    tuition_run.status = 'FULL'
tuition_run.save()

COMMIT

# Notify student
send_email(student, 'TUITION_BOOKING_CONFIRMED', {
    'title': tuition_run.title,
    'start': tuition_run.start_datetime,
    'amount': tuition_run.price
})

return booking

```

8.3 Minimum Class Size Cancellation

python

```

def check_tuition_minimum_class_size():
    """
    Cron job: Check tuition runs approaching start date.
    Cancel if below minimum class size.
    """

    # Find runs starting in 7 days that are still OPEN
    cutoff_date = date.today() + timedelta(days=7)

    runs_to_check = TuitionRun.query.filter(
        start_datetime__date=cutoff_date,
        status='OPEN'
    ).all()

    for run in runs_to_check:
        if run.current_enrollments < run.min_class_size:
            cancel_tuition_run(run)

def cancel_tuition_run(tuition_run):
    """
    Cancel tuition run and refund all students.
    """

    BEGIN TRANSACTION

    # Update run status
    tuition_run.status = 'CANCELLED'
    tuition_run.save()

    # Get all confirmed bookings
    bookings = TuitionBooking.query.filter(
        tuition_run_id=tuition_run.id,
        status='CONFIRMED'
    ).all()

    for booking in bookings:
        student = booking.student
        wallet = student.wallet

        # Credit wallet
        wallet_txn = WalletTransaction.create(
            wallet_id=wallet.id,
            type='CREDIT',
            amount=booking.amount_paid,
            available_before=wallet.available_balance,
            available_after=wallet.available_balance + booking.amount_paid,
            ref_type='TUITION_SESSION',
            ref_id=booking.id,

```

```
status='POSTED',
note=f'Refund: Tuition cancelled (insufficient enrollment)'
)

# Update wallet
wallet.available_balance += booking.amount_paid
wallet.save()

# Cancel booking
booking.status = 'CANCELLED'
booking.save()

# Notify student
send_email(student, 'TUITION_CANCELLED_REFUNDED', {
    'title': tuition_run.title,
    'refund_amount': booking.amount_paid,
    'reason': 'Minimum class size not reached'
})
```

COMMIT

```
log_event('TUITION_RUN_CANCELLED', tuition_run.id,
    f'Only {tuition_run.current_enrollments}/{tuition_run.min_class_size} enrolled')
```

9. BUSINESS RULES & VALIDATION

9.1 Pathway-Based Access Control

```
python
```

```
# Centralized pathway validation decorator
```

```
def requires_pathway(*allowed_pathways):
```

```
    """
```

```
    Decorator to enforce pathway requirements on endpoints.
```

```
Usage:
```

```
    @requires_pathway('FULL_TIME')
```

```
    def create_enrollment(...):
```

```
    """
```

```
def decorator(func):
```

```
    def wrapper(*args, **kwargs):
```

```
        student = kwargs.get('student') or args[0]
```

```
        # Check pathway chosen
```

```
        if not student.study_pathway:
```

```
            raise HTTPException(403,
                                code='PATHWAY_REQUIRED',
                                message='Please choose your study pathway first',
                                action_url='/me/study-pathway'
                                )
```

```
        # Check pathway matches
```

```
        if student.study_pathway not in allowed_pathways:
```

```
            raise HTTPException(403,
                                code='PATHWAY_NOT_ELIGIBLE',
                                message=f'This service requires {" or ".join(allowed_pathways)} pathway'
                                )
```

```
        return func(*args, **kwargs)
```

```
    return wrapper
```

```
    return decorator
```

```
# Usage examples
```

```
@app.post('/full-time/enrollments')
```

```
@requires_pathway('FULL_TIME')
```

```
def create_fulltime_enrollment(student: Student, programme_id: int):
```

```
    # ... implementation
```

```
@app.post('/exam-bookings')
```

```
@requires_pathway('EXAM_ONLY')
```

```
def create_exam_booking(student: Student, module_code: str):
```

```
    # ... implementation
```

```
@app.post('/pool-memberships')
```

```
@requires_pathway('EXAM_ONLY') # ⚠️ CRITICAL - Only exam-only can pool
```

```
def join_pool(student: Student, pool_id: int):
```

```
# ... implementation
```

```
@app.post('/tuition-bookings')
```

```
@requires_pathway('EXAM_ONLY', 'MODULAR')
```

```
def book_tuition(student: Student, tuition_run_id: int):
```

```
# ... implementation
```

9.2 Wallet Balance Validation

```
python
```

```
def validate_sufficient_balance(wallet, amount, purpose='booking'):
```

```
    """
```

```
    Standard wallet balance check.
```

```
    Raises InsufficientFundsError if not enough.
```

```
    """
```

```
    if wallet.available_balance < amount:
```

```
        shortage = amount - wallet.available_balance
```

```
        raise InsufficientFundsError(
```

```
            message=f'Insufficient wallet balance for {purpose}',
```

```
            required=float(amount),
```

```
            available=float(wallet.available_balance),
```

```
            shortage=float(shortage),
```

```
            action='topup',
```

```
            topup_url='/me/wallet/topup'
```

```
        )
```

```
    return True
```

9.3 Payment Schedule Generation Rules

```
python
```

```

def generate_fulltime_milestones(enrollment, programme):
    """
    Generates payment milestones according to rules:
    Year 1: 40% seat / 30% sem1 / 30% sem2
    Year 2+: 50% sem1 / 50% sem2
    """
    milestones = []

    for year_num in range(1, programme.duration_years + 1):
        academic_year = get_academic_year(programme.id, year_num)
        year_fee = academic_year.year_fee_amount or programme.year_fee_amount

        if year_num == 1:
            # Year 1 split
            milestones.append({
                'type': 'SEAT_CONFIRMATION',
                'year': 1,
                'percent': 40.0,
                'amount': year_fee * 0.40,
                'due_date': enrollment.created_at.date()
            })
            milestones.append({
                'type': 'SEM1_DUE',
                'year': 1,
                'percent': 30.0,
                'amount': year_fee * 0.30,
                'due_date': academic_year.semester1_start_date - timedelta(days=7)
            })
            milestones.append({
                'type': 'SEM2_DUE',
                'year': 1,
                'percent': 30.0,
                'amount': year_fee * 0.30,
                'due_date': academic_year.semester2_start_date - timedelta(days=7)
            })
        else:
            # Year 2+ split
            milestones.append({
                'type': 'SEM1_DUE',
                'year': year_num,
                'percent': 50.0,
                'amount': year_fee * 0.50,
                'due_date': academic_year.semester1_start_date - timedelta(days=7)
            })
            milestones.append({
                'type': 'SEM2_DUE',

```

```
'year': year_num,  
'percent': 50.0,  
'amount': year_fee * 0.50,  
'due_date': academic_year.semester2_start_date - timedelta(days=7)  
})
```

```
return milestones
```

9.4 Critical Business Rules Summary

Rule	Enforcement	Impact
Pathway must be chosen	All booking endpoints check	403 if not chosen
Pathway locked after selection	UI + backend prevents change	Admin override only
Wrong pathway rejected	Endpoint-level validation	403 with clear message
Pooling only for EXAM_ONLY	Every pool endpoint validates	Hard block for others
100% upfront for exam/modular/tuition	No partial payments allowed	Wallet must have full amount
Milestone-based for full-time	Generated at enrollment	Cannot skip seat payment
Wallet required for all payments	No direct external payments	Single source of truth
Tuition min class size	Auto-cancel + refund	Protects both students and school

10. API ENDPOINTS COMPLETE

10.1 Pathway Management

POST /api/me/study-pathway

- **Auth:** Student
- **Purpose:** First-time pathway selection (locks immediately)
- **Body:**

```
json  
  
{  
  "study_pathway": "EXAM_ONLY"  
}
```

- **Response:**

json

```
{
  "success": true,
  "study_pathway": "EXAM_ONLY",
  "locked_at": "2026-02-23T10:30:00Z",
  "message": "Pathway locked. You cannot change this."
}
```

- **Errors:**
 - 400: Already chosen
 - 400: Invalid pathway value

POST /api/admin/students/:id/study-pathway

- **Auth:** Admin
- **Purpose:** Override student pathway (rare)
- **Body:**

json

```
{
  "study_pathway": "MODULAR",
  "reason": "Student request due to schedule conflict",
  "mode": "MIGRATE_CANCEL_AND_CREDIT"
}
```

- **Modes:**
 - `BLOCK_IF_BOOKINGS`: Fail if active bookings exist
 - `MIGRATE_CANCEL_AND_CREDIT`: Cancel bookings, refund wallet
- **Response:** Updated student + audit log entry

10.2 Wallet Endpoints

GET /api/me/wallet

- **Auth:** Student
- **Response:**

json

```
{
  "wallet_id": 123,
  "available_balance": 450.00,
  "reserved_balance": 300.00,
  "total_balance": 750.00,
  "currency": "EUR",
  "recent_transactions": [
    {
      "id": 5001,
      "type": "CAPTURE",
      "amount": -300.00,
      "description": "Pool A - M1 exam seat",
      "created_at": "2026-02-20T14:30:00Z"
    }
  ]
}
```

POST /api/me/wallet/topup-request

- **Auth:** Student
- **Purpose:** Initiate top-up (student uploads proof)
- **Body:**

```
json

{
  "amount": 500.00,
  "proof_document": "base64_or_file_upload",
  "payment_method": "bank_transfer"
}
```

- **Response:** Pending top-up record + bank details

POST /api/admin/wallet/:student_id/post-topup

- **Auth:** Admin
- **Purpose:** Verify and post top-up
- **Body:**

```
json
```

```
{  
  "amount": 500.00,  
  "proof_doc_id": 789,  
  "note": "Verified in bank account"  
}
```

- **Effect:** Credits wallet, creates transaction

POST /api/admin/wallet/:student_id/adjust

- **Auth:** Admin
- **Purpose:** Manual adjustment (credit/debit)
- **Body:**

```
json  
  
{  
  "type": "CREDIT",  
  "amount": 100.00,  
  "reason": "Compensation for system error"  
}
```

GET /api/me/wallet/transactions

- **Auth:** Student
- **Query Params:**
 - `from_date`, `to_date`, `type`, `page`, `per_page`
- **Response:** Paginated transaction history

10.3 Full-Time Programme Endpoints

GET /api/catalog/full-time-programmes

- **Auth:** Student (pathway: FULL_TIME)
- **Response:** List of available programmes

POST /api/full-time/enrollments

- **Auth:** Student (pathway: FULL_TIME)
- **Body:**

```
json
```

```
{
  "programme_id": 1
}
```

- **Effect:**
 - Creates enrollment (status: PENDING_CONFIRMATION)
 - Generates all milestones
- **Response:** Enrollment + milestone list

GET /api/full-time/enrollments/:id/milestones

- **Auth:** Student (own enrollment)
- **Response:**

```
json

{
  "enrollment_id": 5,
  "programme_name": "4-Year Full-Time",
  "current_year": 1,
  "milestones": [
    {
      "id": 101,
      "year": 1,
      "type": "SEAT_CONFIRMATION",
      "amount_due": 3200.00,
      "percent": 40.0,
      "due_date": "2026-03-01",
      "status": "DUE",
      "can_pay": true
    },
    {
      "id": 102,
      "year": 1,
      "type": "SEM1_DUE",
      "amount_due": 2400.00,
      "percent": 30.0,
      "due_date": "2026-08-15",
      "status": "DUE",
      "can_pay": false
    }
  ]
}
```

POST /api/full-time/milestones/:id/pay

- **Auth:** Student
- **Body:** (optional confirmation)

```
json

{
  "confirm": true
}
```

- **Effect:**
 - Captures from wallet
 - Marks milestone PAID
 - If seat confirmation: enrollment → ACTIVE
- **Response:** Updated milestone + wallet balance

10.4 Exam-Only Endpoints

GET /api/catalog/exam-modules

- **Auth:** Student (pathway: EXAM_ONLY)
- **Response:** List of EASA modules

POST /api/exam-bookings

- **Auth:** Student (pathway: EXAM_ONLY)
- **Body:**

```
json

{
  "module_code": "M1",
  "event_window_id": 5,
  "booking_type": "INDIVIDUAL"
}
```

- **Effect:**
 - Captures €520 from wallet (100% upfront)
 - Creates CONFIRMED booking
- **Response:** Booking details

Pooling Endpoints *[All pooling endpoints from previous spec with pathway validation added]*

- Every endpoint starts with:

```
python

if student.study_pathway != 'EXAM_ONLY':
    raise PathwayError()
```

10.5 Modular Package Endpoints

GET /api/catalog/modular-packages

- **Auth:** Student (pathway: MODULAR)
- **Response:** Available packages

POST /api/modular-enrollments

- **Auth:** Student (pathway: MODULAR)
- **Body:**

```
json

{
  "package_id": 3
}
```

- **Effect:**
 - Captures full price from wallet
 - Activates package
 - Grants module access
- **Response:** Enrollment details

10.6 Tuition Session Endpoints

GET /api/catalog/tuition-runs

- **Auth:** Student (pathway: EXAM_ONLY or MODULAR)
- **Query Params:** `module_tag`, `upcoming_only`
- **Response:** Available tuition sessions

POST /api/tuition-bookings

- **Auth:** Student (pathway: EXAM_ONLY or MODULAR)
- **Body:**

```
json
```

```
{  
  "tuition_run_id": 42  
}
```

- **Effect:**
 - Captures price from wallet
 - Confirms booking
 - Increments run enrollment count
 - **Response:** Booking confirmation
-

11. WALLET TRANSACTION FLOWS

11.1 Complete Transaction Patterns

Pattern 1: Simple Capture (Exam/Modular/Tuition)

```
[Student] → "Book X" → [System validates] → [Wallet CAPTURE] → [Booking CONFIRMED]
```

Pattern 2: Reserve → Capture (Pooling)

```
[Student] → "Join Pool" → [Wallet RESERVE] → [Pool pending]  
→ [Pool reaches 25] → [Wallet CAPTURE from reserved] → [Membership CONFIRMED]
```

Pattern 3: Reserve → Release (Pool Fails)

```
[Student] → "Join Pool" → [Wallet RESERVE] → [Pool pending]  
→ [T-21, pool <25] → [Wallet RELEASE to available] → [Membership FAILED]
```

Pattern 4: Capture → Credit (Cancellation)

```
[Admin] → "Cancel tuition" → [Wallet CREDIT] → [Booking CANCELLED]
```

11.2 Atomic Transaction Template

```
python
```

```

def wallet_capture_transaction(wallet_id, amount, ref_type, ref_id, note):
    """
    Standard wallet capture transaction.
    ALWAYS use this for consistency.
    """
    wallet = Wallet.get(wallet_id)

    # Validate
    if wallet.available_balance < amount:
        raise InsufficientFundsError()

    BEGIN TRANSACTION

    # Create transaction record FIRST
    txn = WalletTransaction.create(
        wallet_id=wallet_id,
        type='CAPTURE',
        amount=amount,
        available_before=wallet.available_balance,
        available_after=wallet.available_balance - amount,
        reserved_before=wallet.reserved_balance,
        reserved_after=wallet.reserved_balance,
        ref_type=ref_type,
        ref_id=ref_id,
        status='POSTED',
        created_by='system',
        note=note
    )

    # Update wallet
    wallet.available_balance -= amount
    wallet.updated_at = NOW()
    wallet.save()

    COMMIT

    return txn

```

11.3 Reconciliation Queries

sql

-- Check wallet balance accuracy

SELECT

w.id AS wallet_id,

w.student_id,

w.available_balance AS system_balance,

COALESCE(SUM(

CASE

WHEN wt.type IN ('TOP_UP', 'CREDIT', 'RELEASE') THEN wt.amount

WHEN wt.type IN ('CAPTURE', 'DEBIT') THEN -wt.amount

ELSE 0

END

), 0) AS calculated_balance,

w.available_balance - COALESCE(SUM(

CASE

WHEN wt.type IN ('TOP_UP', 'CREDIT', 'RELEASE') THEN wt.amount

WHEN wt.type IN ('CAPTURE', 'DEBIT') THEN -wt.amount

ELSE 0

END

), 0) AS discrepancy

FROM wallets w

LEFT JOIN wallet_transactions wt ON w.id = wt.wallet_id AND wt.status = 'POSTED'

GROUP BY w.id

HAVING ABS(w.available_balance - COALESCE(SUM(

CASE

WHEN wt.type IN ('TOP_UP', 'CREDIT', 'RELEASE') THEN wt.amount

WHEN wt.type IN ('CAPTURE', 'DEBIT') THEN -wt.amount

ELSE 0

END

), 0)) > 0.01;

12. USER INTERFACE SPECIFICATIONS

12.1 First Login Pathway Wizard

Screen: Choose Your Study Path

Dashboard	[Wallet: €450.00 ]	
Study Path: FULL-TIME (4-Year)	 Locked	
 My Programme		
└─ Current: Year 1, Semester 1		
└─ Status: ACTIVE 		
└─ [View Timetable] [Attendance] [Records]		
 Payment Milestones		
└─ Year 1 Seat (40%): PAID 		
└─ Year 1 Sem 1 (30%): DUE €2,400		
└─ Year 1 Sem 2 (30%): Upcoming		
[Pay Next Milestone →]		
 My Modules		
 Calendar		

Exam-Only Student Dashboard:

Dashboard	[Wallet: €700.00 🔔]
Study Path: EXAM-ONLY 🔒 Locked	
🎯 Book Exams	
└ Individual Seats (€520)	
└ Join Exam Pool (€300) - Save €220!	
[Browse Upcoming Events →]	
👥 My Pools	
Pool A (Jun 2026): CONFIRMED ✓	
• Module: M1 - Mathematics	
• Date: Jun 23, 9:00 AM	
[View Details]	
Pool B (Jun 2026): RESERVED (19/25)	
• Module: M7 - Maintenance	
• Needs 6 more candidates	
[Withdraw]	
📅 My Exam Schedule	
📊 Results	

Modular Student Dashboard:

Dashboard	[Wallet: €200.00 	
Study Path: MODULAR  Locked		
 My Package		
Foundation 4-Pack (Active)		
└─ Modules: M1, M2, M7, M8		
└─ Valid until: Feb 23, 2027		
└─ [Access Materials]		
 Learning Progress		
└─ M1: 75% complete		
└─ M2: 30% complete		
└─ M7: Not started		
 Optional Tuition Sessions		
[Browse Available Classes →]		

12.3 Wallet Interface

Wallet Page:

My Wallet

[Top Up 💰]

Available Balance: €450.00

Reserved (pending): €300.00

Total: €750.00

Reserved Breakdown:

• Pool B (M7): €300.00

Recent Transactions

[View All]

Feb 20 CAPTURE -€300.00 Pool A - M1 seat

Feb 18 RESERVE -€300.00 Pool B - M7 (pending)

Feb 15 TOP_UP +€1000.00 Bank transfer

[Download Statement]

12.4 Booking Confirmation Modal (Universal)

Confirm Booking

[×]

You're about to book:

Pool Seat - Jun 2026 Event

Module: M1 - Mathematics

Amount to be deducted: €300.00

Your wallet after: €150.00

i

Funds will be reserved until pool confirms
(needs 25 candidates total)

☐ I understand this will deduct from my wallet

[Cancel]

[Confirm Booking ✓]

13. TESTING & DEPLOYMENT

13.1 Critical Test Cases

Pathway System:

- ☐ Student cannot access any booking until pathway chosen
- ☐ Student cannot change pathway after lock
- ☐ Wrong pathway bookings return 403 with clear message
- ☐ Admin pathway change requires reason + audit log
- ☐ Admin pathway change with bookings handles correctly

Wallet System:

- ☐ Top-up increases available balance correctly
- ☐ Capture decreases available balance atomically
- ☐ Reserve moves available → reserved
- ☐ Capture from reserved works correctly
- ☐ Release moves reserved → available
- ☐ Concurrent transactions don't double-spend
- ☐ Balance reconciliation query shows no discrepancies

Full-Time:

- ☐ Enrollment creates milestones (40/30/30 Y1, 50/50 Y2+)
- ☐ Enrollment stays PENDING_CONFIRMATION until seat paid
- ☐ Paying seat milestone activates enrollment
- ☐ Cannot pay milestone without sufficient wallet balance

Exam-Only:

- ☐ Individual booking captures €520 immediately
- ☐ Pool booking requires EXAM_ONLY pathway
- ☐ Non exam-only students blocked from pooling (403)
- ☐ Pool reserve/capture/release flows work correctly

Modular:

- ☐ Package purchase captures 100% upfront
- ☐ Grants access to included modules

Tuition:

- ☐ Booking captures 100% upfront
- ☐ Min class size cancellation refunds correctly
- ☐ EXAM_ONLY and MODULAR can book
- ☐ FULL_TIME blocked unless configured

13.2 Integration Test Scenarios

Scenario 1: Exam-Only Student Complete Journey

1. Register → First login
2. Choose EXAM_ONLY pathway (locked)
3. Top up wallet €1,000
4. Join Pool A for M1 (€300 reserved)
5. Pool A reaches 25 → €300 captured, booking confirmed
6. Join Pool B for M7 (€300 reserved)
7. Pool B fails → €300 released back
8. Book individual seat for M8 (€520 captured)
9. Attend exams, complete

Scenario 2: Full-Time Student Payment Timeline

1. Register → Choose FULL_TIME
2. Enroll in 4-year programme
3. Top up wallet €3,500
4. Pay Year 1 seat confirmation 40% (€3,200) → enrollment ACTIVE
5. Pay Year 1 Sem 1 30% (€2,400) before semester start
6. Pay Year 1 Sem 2 30% (€2,400) before semester start
7. Year 2 milestones generated automatically

Scenario 3: Modular Student Package Purchase

1. Register → Choose MODULAR
2. Top up wallet €1,900
3. Purchase 4-seat bundle (€1,900 captured)
4. Access granted to M1, M2, M7, M8
5. Book tuition session for M1 (€150 captured)
6. Attend tuition, complete modules

13.3 Deployment Plan

Phase 1: Core Infrastructure (Week 1-2)

- ☐ Database schema deployed
- ☐ Wallet system implemented
- ☐ Pathway selection wizard

☐ Admin override functions

Phase 2: Full-Time System (Week 3-4)

- ☐ Programme management
- ☐ Milestone generation
- ☐ Payment processing

Phase 3: Exam-Only + Pooling (Week 5-7)

- ☐ Individual exam bookings
- ☐ Pooling system integration
- ☐ Pathway gates enforced

Phase 4: Modular + Tuition (Week 8-9)

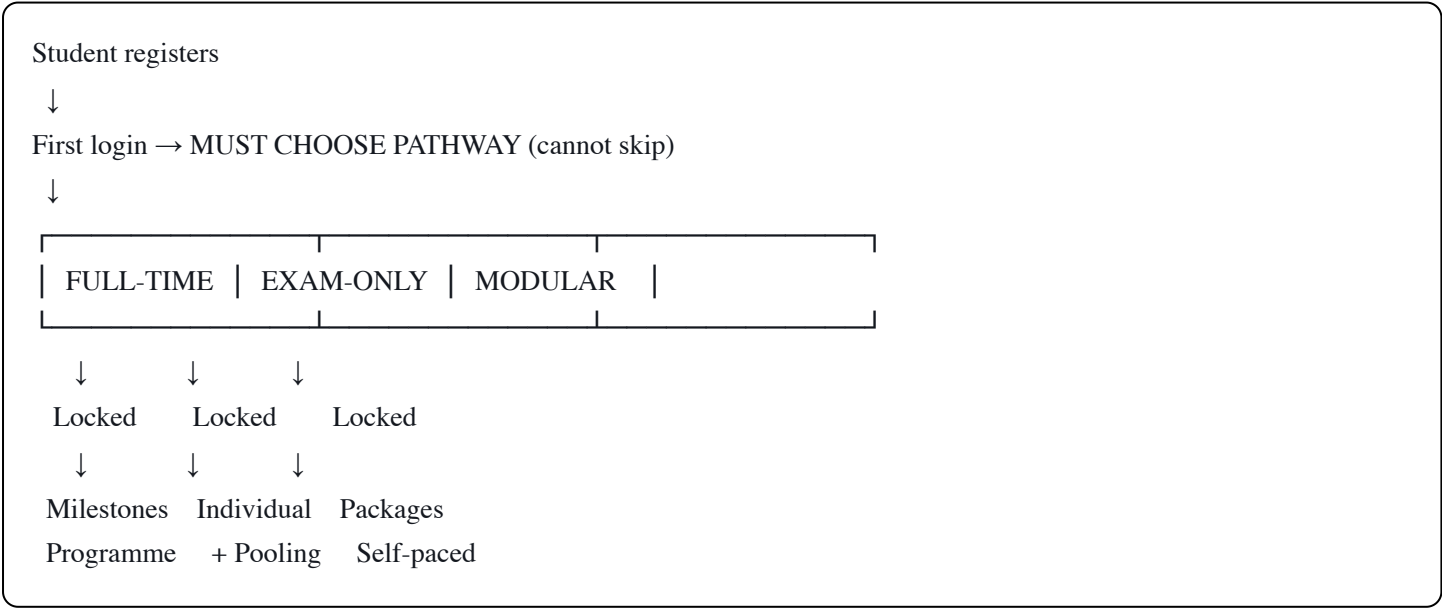
- ☐ Package system
- ☐ Tuition booking
- ☐ Min class size automation

Phase 5: Testing & Launch (Week 10-12)

- ☐ Integration testing
- ☐ User acceptance testing
- ☐ Soft launch (limited users)
- ☐ Full launch

APPENDIX: QUICK REFERENCE

Pathway Decision Tree



Wallet Transaction Quick Ref

Type	Effect	Use
TOP_UP	+available	Add funds
RESERVE	avail→reserve	Pool join
CAPTURE	-available or -reserve	Booking confirm
RELEASE	reserve→avail	Pool fail
CREDIT	+available	Refund

Payment Rules Quick Ref

Product	Payment	Pathway
Full-time	40/30/30, 50/50	FULL_TIME
Individual exam	100% upfront (€520)	EXAM_ONLY
Pool exam	100% upfront (€300)	EXAM_ONLY only
Modular	100% upfront	MODULAR
Tuition	100% upfront	EXAM_ONLY, MODULAR

END OF INTEGRATED SPECIFICATION

For Developer Questions: development@aerojet-academy.com

Version History:

- v1.0: Initial pooling spec
- v2.0: Integrated pathways + wallet + all booking types